LAB 2: SEQUENTIAL LOGIC & MEMORY HIERARCHY

NAND2Tetris HDL Submission

Student	RAVINDER
Lab	Sequential Logic & Memory Hierarchy
Components	RAM512, RAM1K, RAM16K, Program Counter (PC)
Platform	NAND2Tetris HDL

Contents: HDL listings, memory hierarchy and address decomposition, latency comparison, PC state-transition table, timing behavior, and cache hierarchy relation.

Q5. RAM512, RAM1K and RAM16K

The RAM hierarchy is constructed by decoding the high-order address bits to select a lower-level RAM block. The low-order address bits are passed to the selected block. The output is selected using the same high-order address bits.

RAM512.hdl

```
CHIP RAM512 {
    IN in[16], load, address[9];
    OUT out[16];

    PARTS:
    DMux8Way(in=load, sel=address[6..8],
        a=load0, b=load1, c=load2, d=load3,
        e=load4, f=load5, g=load6, h=load7);

    RAM64(in=in, load=load0, address=address[0..5], out=out0);
    RAM64(in=in, load=load1, address=address[0..5], out=out1);
    RAM64(in=in, load=load2, address=address[0..5], out=out2);
    RAM64(in=in, load=load3, address=address[0..5], out=out3);
    RAM64(in=in, load=load4, address=address[0..5], out=out4);
    RAM64(in=in, load=load5, address=address[0..5], out=out5);
    RAM64(in=in, load=load6, address=address[0..5], out=out6);
    RAM64(in=in, load=load7, address=address[0..5], out=out7);

    Mux8Way16(
        a=out0, b=out1, c=out2, d=out3,
        e=out4, f=out5, g=out6, h=out7,
        sel=address[6..8], out=out
    );
}
```

RAM1K.hdl

```
CHIP RAM1K {
    IN in[16], load, address[10];
    OUT out[16];

    PARTS:
    DMux2Way(in=load, sel=address[9],
        a=load0, b=load1);

    RAM512(
        in=in,
        load=load0,
        address=address[0..8],
        out=out0
    );

    RAM512(
        in=in,
        load=load1,
        address=address[0..8],
        out=out1
    );

    Mux16(
        a=out0,
        b=out1,
        sel=address[9],
        out=out
    );
}
```

RAM16K.hdl

```
CHIP RAM16K {
    IN in[16], load, address[14];
    OUT out[16];

    PARTS:
    DMux4Way(in=load, sel=address[12..13],
        a=load0, b=load1, c=load2, d=load3);

    RAM4K(
        in=in,
        load=load0,
        address=address[0..11],
        out=out0
    );

    RAM4K(
        in=in,
```

```
        load=load1,
        address=address[0..11],
        out=out1
    );

    RAM4K(
        in=in,
        load=load2,
        address=address[0..11],
        out=out2
    );

    RAM4K(
        in=in,
        load=load3,
        address=address[0..11],
        out=out3
    );

    Mux4Way16(
        a=out0, b=out1, c=out2, d=out3,
        sel=address[12..13], out=out
    );
}
```

Address decomposition

RAM	Capacity	Address bits	Block select	Within-block address
RAM512	512 × 16	9	address[8..6]	address[5..0]
RAM1K	1024 × 16	10	address[9]	address[8..0]
RAM16K	16384 × 16	14	address[13..12]	address[11..0]

Memory hierarchy

RAM16K → 4 × RAM4K → 16 × RAM1K → 32 × RAM512 → 256 × RAM64, with each lower level continuing down the NAND2Tetris memory hierarchy to Register cells. The high-order address bits select the block while the low-order bits select the location inside that block.

Read/write latency comparison

RAM	Read latency	Write latency	Observation
RAM512	0 extra clock cycles	1 clock edge	Leaf registers update on clock
RAM1K	0 extra clock cycles	1 clock edge	Same synchronous storage behavior
RAM16K	0 extra clock cycles	1 clock edge	Larger decode/mux hierarchy

Note: In NAND2Tetris, the RAM hierarchy does not introduce additional sequential clock cycles for larger capacities. Writes occur on the clock edge, while the stored output is available after the state update. Physical propagation delay and simulator wall-clock runtime can increase with hierarchy depth, but that is distinct from clock-cycle latency.

Q6. Program Counter (PC)

The PC stores a 16-bit address and chooses the next value according to the priority reset > load > increment > hold.

PC.hdl

```
CHIP PC {
    IN in[16], load, inc, reset;
    OUT out[16];

    PARTS:
    Inc16(in=out, out=incOut);

    Mux16(a=out, b=incOut, sel=inc, out=incResult);
    Mux16(a=incResult, b=in, sel=load, out=loadResult);
    Mux16(a=loadResult, b=false, sel=reset, out=next);

    Register(in=next, load=true, out=out);
}
```

Complete state-transition table

reset	load	inc	Next PC	Priority / action
0	0	0	PC	Hold current value
0	0	1	PC + 1	Increment
0	1	0	in	Load
0	1	1	in	Load overrides increment
1	0	0	0	Reset
1	0	1	0	Reset overrides increment
1	1	0	0	Reset overrides load
1	1	1	0	Reset has highest priority

PC timing diagram / cycle behavior

```
Clock:    ↑       ↑       ↑       ↑       ↑
Cycle:    0       1       2       3       4
reset:    0       0       0       1       0
load:     0       1       0       0       0
inc:      1       0       1       0       0
in:       A       B       C       D       E
PC:       X     X+1       B     B+1       0

At each rising clock edge, the selected next value is stored in the Register.
```

Example interpretation: with reset=0, load=0, inc=1, the PC increments at the clock edge. With load=1, the input value is stored instead. When reset=1, the PC becomes 0 regardless of the other two control signals.

Relation to Real L1/L2 Cache Hierarchy

NAND2Tetris memory	Real computer memory hierarchy	Key idea
Register / small RAM	CPU registers / L1 cache	Very fast, small storage close to computation
RAM hierarchy	L2/L3 cache and main memory	Larger capacity with greater access complexity
Address decoding	Cache indexing/tag lookup	Address bits determine where data is selected
Mux / selection logic	Cache way selection	Chooses the requested data source

The NAND2Tetris hierarchy demonstrates the same fundamental engineering trade-off found in real systems: smaller storage can be accessed more directly, while larger storage requires more selection and hierarchy. Real L1/L2 caches additionally use tags, valid bits, replacement policies, and cache lines, which are outside the basic RAM chips implemented here.

Conclusion

RAM512, RAM1K and RAM16K demonstrate hierarchical memory construction through address decomposition, demultiplexing and multiplexing. The PC demonstrates synchronous state storage and prioritized control signals. Together, these components illustrate the core principles of sequential logic and hierarchical memory used by larger computer systems.